

SERVICE ACTUEL · WHITELISTER

Règles réseau entrantes et découverte des IP de sortie

6 capacités analysées

services/whitelister · Python, FastAPI, SQLAlchemy asynchrone et PostgreSQL ; client Kubernetes, ConfigMaps et agent de découverte ; JWT/JWKS et limitation de débit.

Permettre à l'exploitation de gérer les CIDR admis par l'ingress et d'identifier les adresses vues par les services externes. La découverte d'IP de sortie n'est pas une politique d'autorisation des destinations sortantes.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Egress, IAM, Audit, Maintenance.

Périmètre de lecture

- Routes, schémas CIDR et egress ; services whitelist, Kubernetes, egress et audit ; middleware et boucle de démarrage.
- Agent de découverte et contrats chart repérés ; aucune opération Kubernetes exécutée.

Objets et cycle de vie

- WhitelistRule porte CIDR, description et flags protected/init. AuditLog enregistre action et attribution.
- Les observations EgressEntry sont horodatées par nœud, distinctes des règles de destinations sortantes.

Dépendances du service

- PostgreSQL et Kubernetes Core API pour les ConfigMaps.
- Ingress NGINX, agents par nœud, service d'écho et issuer/JWKS configuré.

Fonctionnalités et compatibilité à préserver

WL-01 Cible non qualifiée

Créer et éditer une liste de CIDR admis

COMPORTEMENT OBSERVÉ

L'API normalise les CIDR, crée une règle ou un lot d'au plus cent, puis expose liste paginée et détail. Les règles manuelles ont init=false ; protected peut être déclaré à la création. Le lot écrit ses règles dans une transaction et refuse une collision d'unicité. PUT et PATCH modifient CIDR ou description ; une règle protégée ne peut être modifiée. Les écritures déclenchent ensuite la synchronisation Kubernetes.

UTILISATEURS ET RÔLES

Administrateurs réseau.

INTERFACES DANS LE CODE

POST/GET /api/v1/whitelist ; POST /api/v1/whitelist/bulk ; GET/PUT/PATCH /api/v1/whitelist/{rule_id}.

DONNÉES ET ÉTATS

WhitelistRule avec CIDR normalisés, description, protected, init et dates en PostgreSQL.

DÉPENDANCES ET ÉCHANGES

PostgreSQL et contrôleur de ConfigMap ingress.

COMPORTEMENT CIBLE PROPOSÉ

Un module de politique réseau Rust conserve les règles désirées, leur version et leur provenance. Le contrôleur Kubernetes applique une génération distincte, de sorte que l'API distingue accepté et appliqué.

MIGRATION DE CETTE CAPACITÉ

Importer les règles manuelles et celles de configuration sans changer leur protection. Conserver normalisation, limites de lot et erreurs, puis basculer l'unique contrôleur de ConfigMap.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lot contenant un doublon n'écrit aucune règle. Une règle protected ne peut changer par PATCH. La réponse indique la génération effective lorsque la projection réseau est en retard.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/schemas/whitelist.py:13](#) Normalisation du contrat.

[monorepo](#) · [services/whitelister/src/whitelister/services/whitelist_service.py:157](#) Transaction de lot et collision.

[monorepo](#) · [services/whitelister/src/whitelister/services/whitelist_service.py:305](#) Protection et mise à jour.

WL-02 **Cible non qualifiée**

Refuser le retrait de sa propre plage et des règles protégées**COMPORTEMENT OBSERVÉ**

La suppression distingue introuvable, protégé et auto-exclusion. Une règle protégée donne un refus. Si l'adresse cliente se trouve dans l'un des CIDR de la règle, la suppression est aussi refusée et auditée. L'adresse vient de l'extraction du contexte réseau/proxy. Le code ignore une adresse impossible à parser dans ce contrôle ; le résultat dépend donc aussi de la confiance accordée aux headers proxy.

UTILISATEURS ET RÔLES

Administrateurs modifiant la liste d'accès réseau.

INTERFACES DANS LE CODE

DELETE /api/v1/whitelist/{rule_id} ; WhitelistService.delete_rule.

DONNÉES ET ÉTATS

Règle et audit de tentative bloquée ; aucune suppression pour les deux refus métier.

DÉPENDANCES ET ÉCHANGES

Extraction de l'IP par le proxy et journal d'audit.

COMPORTEMENT CIBLE PROPOSÉ

Le contrôleur Rust valide l'identité réseau de l'opérateur et la projection simulée avant retrait. Une procédure de secours bornée peut traiter un changement volontaire sans supprimer les garanties de l'API ordinaire.

MIGRATION DE CETTE CAPACITÉ

Reprendre le contrat de refus et qualifier le calcul d'adresse derrière l'ingress réel. Ajouter un plan de changement qui évalue l'accès d'administration restant.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un opérateur ne peut retirer la plage qui contient son adresse vérifiée. Un header de provenance forgé ne contourne pas ce contrôle. La tentative refusée reste visible dans l'audit.

Références de l'analyse

[monorepo · services/whitelister/src/whitelister/services/whitelist_service.py:226](#) Protections et résultats de suppression.

[monorepo · services/whitelister/src/whitelister/api/routes.py:346](#) Correspondance des refus vers les réponses HTTP.

WL-03 **Cible non qualifiée****Projeter les règles vers l'ingress et réinitialiser la configuration de démarrage****COMPOTEMENT OBSERVÉ**

Le service lit toutes les règles, joint les CIDR puis met à jour une clé d'une ConfigMap Kubernetes. La synchronisation réessaie trois fois avec attente croissante ; après le dernier échec, elle journalise et ne remonte pas l'exception. Une création peut donc être enregistrée en base sans preuve que l'ingress l'a appliquée. POST sync relance cette projection. Au démarrage, les règles init sont supprimées et recrées depuis la configuration, y compris si elles étaient protégées.

UTILISATEURS ET RÔLES

Exploitation et automatisation du déploiement.

INTERFACES DANS LE CODE

POST /api/v1/whitelist/sync ; sync_to_kubernetes ; reset_init_rules ; KubernetesService.update_configmap.

DONNÉES ET ÉTATS

État désiré PostgreSQL et ConfigMap appliquée ; règles de démarrage distinctes des règles manuelles.

DÉPENDANCES ET ÉCHANGES

API Kubernetes et ingress NGINX consommateur de la ConfigMap.

COMPOTEMENT CIBLE PROPOSÉ

Un reconciler Rust garde génération désirée, génération observée et erreur durable. L'initialisation calcule un diff et ne traverse pas un état intermédiaire non voulu. Le statut appliqué exige la lecture de la projection effective.

MIGRATION DE CETTE CAPACITÉ

Capturer la ConfigMap et comparer les CIDR normalisés à la base avant transfert. Désactiver l'ancien sync lorsque le nouveau contrôleur devient auteur. Préserver les règles manuelles lors d'un changement de configuration.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne Kubernetes laisse l'opération pending ou failed, sans succès d'application trompeur. Un redémarrage avec la même configuration ne retire pas temporairement les règles attendues.

Références de l'analyse

[monorepo · services/whitelister/src/whitelister/services/whitelist_service.py:57](#) Retries et erreur finale absorbée.

[monorepo · services/whitelister/src/whitelister/services/whitelist_service.py:87](#) Cycle de réinitialisation des règles init.

[monorepo](#) · [services/whitelister/src/whitelister/services/kubernetes_service.py:28](#) Écriture de la clé de ConfigMap.

WL-04 **Cible non qualifiée**

Découvrir les IP de sortie et maintenir leur règle

COMPORTEMENT OBSERVÉ

Des agents par nœud interrogent un service d'écho et publient leurs observations horodatées en ConfigMap. Le service agrège les IP, ajoute les overrides valides et signale les observations périmées, mais les conserve dans l'ensemble. GET egress-ips déduplique les IP et renvoie origine, nœud, date et warnings. Une boucle peut maintenir une règle protégée pour ces IP. Le rendu CIDR ajoute /32 à chaque IP ; la prise en charge IPv6 nécessite donc une vérification spécifique.

UTILISATEURS ET RÔLES

Exploitation et administrateurs d'intégrations externes.

INTERFACES DANS LE CODE

GET /api/v1/egress-ips ; egress_agent ; EgressService.get_all_egress_ips/upsert_egress_rule.

DONNÉES ET ÉTATS

Observations de nœuds en ConfigMap, overrides et règle protégée PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Agents Kubernetes, service d'écho, ConfigMap et boucle de synchronisation.

COMPORTEMENT CIBLE PROPOSÉ

Egress Rust sépare observation réseau et politique des destinations autorisées. Le discovery garde fraîcheur, famille IP et preuve d'observation ; les connecteurs consomment une politique de sortie avec DNS/TLS et borne de périmètre, pas une simple liste d'IP vues.

MIGRATION DE CETTE CAPACITÉ

Porter l'agent sans JVM ou le garder encapsulé, puis publier un contrat IPv4/IPv6 et une politique explicite de péremption. Maintenir les overrides nécessaires pendant le changement de topologie.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une observation périmée est marquée et son effet suit une politique explicite. Une IPv6 produit un préfixe adapté. Une panne du service d'écho ne transforme pas une ancienne IP en observation fraîche.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/egress_agent.py:28](#) Découverte et validation des réponses d'écho.

[monorepo](#) · [services/whitelister/src/whitelister/services/egress_service.py:87](#) Agrégation, overrides et données périmées.

[monorepo](#) · [services/whitelister/src/whitelister/services/egress_service.py:145](#) Règle protégée et construction de CIDR.

WL-05 **Cible non qualifiée**

Authentifier les administrateurs réseau et limiter les requêtes

COMPORTEMENT OBSERVÉ

Quand l'authentification est activée, le middleware exige un bearer vérifié par JWKS, signature, dates, issuer et audience ou azp configurés. Le contrôle de groupes accepte un nom de groupe ou un chemin complet. Les routes exclues et preflights suivent un parcours séparé. Lorsque l'authentification est désactivée, un token

peut être vérifié seulement pour attribution d'audit ; l'absence de token n'est alors pas un refus. Les écritures ont une limite de débit et la découverte d'egress une borne spécifique.

UTILISATEURS ET RÔLES

Administrateurs réseau authentifiés.

INTERFACES DANS LE CODE

JWTAuthMiddleware.dispatch ; verify_access_token ; décorateurs limiter des routes.

DONNÉES ET ÉTATS

Utilisateur vérifié dans request.state ; cache de groupes requis et validation JWKS.

DÉPENDANCES ET ÉCHANGES

OIDC/JWKS, proxy et futur broker IAM.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust fournit une permission d'administration réseau explicite. Le service vérifie issuer, audience, sujet actif et décision ; le mode sans authentification reste une configuration isolée et déclarée, jamais une dégradation automatique.

MIGRATION DE CETTE CAPACITÉ

Remplacer les groupes Keycloak par des relations métier sans perdre l'accès de secours prévu. Mettre à jour l'issuer/JWKS avec les autres services et qualifier les chemins de santé exclus.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un JWT avec mauvaise signature, issuer ou audience est refusé. Un membre d'un groupe de nom voisin n'obtient pas le droit. Une panne du fournisseur ne désactive pas l'authentification.

Références de l'analyse

[monorepo · services/whitelister/src/whitelister/middleware.py:155](#) Contrat de vérification, groupes et mode désactivé.

[monorepo · services/whitelister/src/whitelister/api/routes.py:437](#) Borne spécifique de découverte.

WL-06 Cible non qualifiée

Consulter les audits et distinguer santé de base et état réseau

COMPORTEMENT OBSERVÉ

Les opérations et refus alimentent un journal avec action, IP cliente, utilisateur et règle. Les routes listent avec filtres et pagination puis rendent un détail. Le helper général d'audit journalise et absorbe les erreurs d'écriture ; une action peut donc réussir sans audit durable confirmé. Le health contrôle une requête PostgreSQL, pas l'état effectif de l'ingress. Les métriques Prometheus sont exposées séparément.

UTILISATEURS ET RÔLES

Exploitation, support et auditeurs.

INTERFACES DANS LE CODE

GET /api/v1/audit ; GET /api/v1/audit/{log_id} ; GET /api/v1/health ; /metrics.

DONNÉES ET ÉTATS

AuditLog PostgreSQL et métriques de processus ; pas de génération réseau acquittée dans le contrat lu.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, métriques Prometheus et contrôleur Kubernetes.

COMPORTEMENT CIBLE PROPOSÉ

Audit Rust conserve intention et résultat dans une transaction durable ; le contrôleur rapporte séparément application Kubernetes. La santé distingue API, base, retard de projection et découverte réseau.

MIGRATION DE CETTE CAPACITÉ

Importer l'audit existant avec sa complétude déclarée. Ajouter identifiants de changement et reçus d'application avant de couper l'ancien service.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne de journal n'est pas présentée comme une action intégralement auditée. Une base disponible et un ingress non synchronisé produisent deux états distincts. Un filtre ne révèle pas de secrets réseau ou credentials.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/services/whitelist_service.py:31](#) Écriture d'audit sans remontée d'erreur.

[monorepo](#) · [services/whitelister/src/whitelister/api/routes.py:499](#) Filtres et pagination.

[monorepo](#) · [services/whitelister/src/whitelister/api/routes.py:486](#) Santé limitée à la connexion base.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Gestion des règles désirées, approbation, générations, état de projection et audit durable.
- Politique d'autorisation des destinations pour connecteurs à concevoir séparément de la découverte d'IP.

Plan de données proposé

- Le service actuel ne relaie pas le trafic applicatif. L'ingress applique les CIDR ; les agents observent la sortie réseau.

Découpage du core Rust

- Normaliseur CIDR, planificateur de diff, reconciler Kubernetes et modèle d'observation réseau.
- Adaptateur IAM et journal transactionnel pour les changements.

Identité, autorisation et isolation

- La confiance dans l'IP cliente dépend du proxy ; conserver une attribution vérifiée.
- La découverte ne constitue pas une barrière anti-exfiltration. La cible doit contrôler les destinations et les capacités de chaque run.

Exploitation et reprise

- Surveiller divergence base/ConfigMap, âge des observations et erreurs d'audit.
- Le startup remplace actuellement les règles init ; tester les changements de configuration et les échecs Kubernetes avant remplacement.

Séquence de migration

- Comparer règles en base et projection ingress.
- Déployer un reconciler Rust en observation puis transférer l'écriture de ConfigMap.
- Porter authz, audit et discovery en gardant une procédure de secours bornée.

Écarts et décisions encore ouverts

- Le traitement IPv6 et la conservation des observations périmées doivent être contractualisés.
- Le lot n'a pas exécuté de mutation réseau ni de test d'auto-exclusion.
- Le succès du sync actuel n'est pas une preuve que tous les contrôleurs ingress ont chargé la règle.

Références de l'analyse

[monorepo](#) · [services/whitelister/src/whitelister/main.py:43](#) Boucle de synchronisation des IP de sortie.

[monorepo](#) · [services/whitelister/src/whitelister/main.py:70](#) Initialisation et tâches de fond.